

Conjunction OS

A Foundational Architecture for Cyclic, Distinction-Preserving Computation

Abstract

Conjunction OS is a computational architecture organized around four refusals: it refuses to equate distinct states, it refuses to erase without routing, it refuses to accumulate without closure, and it refuses to treat composition as addition. This paper states the refusals as formal postconditions over a three-part system state, types the core operators with contracts, walks one complete four-phase cycle on concrete data, and proposes four evaluation metrics by which an implementation can be checked. Architectural consequences include a non-monotonic alternative to scalar-loss learning, a three-rail reversibility structure — grounded in the same topology that makes the Tower of Hanoi solvable — that explains the Toffoli gate as a load-bearing primitive of the commitment step, and an HTTP surface with explicit idempotency, atomicity, and crash-recovery semantics.

1. Introduction

An operating system is not, first of all, a piece of software. It is a set of commitments about how distinctions are preserved, how information leaves a boundary, and how time is allowed to close. Every implementation detail follows from those commitments, and every failure mode follows from a place where one of them was quietly waived.

Modern general-purpose computing architectures have made a particular set of choices. A Boolean filter returns true or false and discards everything else. A gradient-descent loop collapses all structural disagreement between a prediction and a target into a single scalar loss. A memory system accumulates state indefinitely and evicts older records when capacity is reached. Each choice is defensible; each is also a decision to let certain distinctions disappear. The cost surfaces later, as unreproducible training dynamics, as silent data drift, as the unbounded monotonicity of a learning curve that has stopped representing anything about its environment.

Conjunction OS takes those costs seriously. Its design question is not *how do we compute efficiently* but rather *what must an architecture refuse to do if it is to avoid silent collapse?* Four refusals follow, and the remainder of this paper treats each as a checkable postcondition rather than a stance.

2. The Four Principles

2.1 The Non-Identity Principle

No two distinct states may be treated as equivalent by any operation of the system. When an operator receives inputs that ought to be distinguished, the correct response is to surface that distinction, not to quietly absorb it. False equivalence is the only catastrophic failure class; every other error is correctable. A system that admits one false identity corrupts every downstream inference that rests on it.

2.2 The Conservative Residue Principle

What a filter rejects is not noise. It is structured residual signal — the portion of the input the filter's criterion could not absorb. Discarding residue is thermodynamically and informationally expensive

(cf. Landauer), and architecturally it is a silent collapse: it equates *rejected* with *absent*. Conjunction OS requires every rejection to have a destination. Residue is routed, never dropped.

2.3 The Cyclic Closure Principle

Computation proceeds in closed cycles that return to a zero condition, not in monotonically accumulating trajectories. A cycle is complete only when its residue has been fully routed and the working state has settled. Open residue at the end of a cycle signals that the wave has not closed; the next cycle must wait. Time, in this architecture, is phasic rather than linear.

2.4 The Non-Additive Composition Principle

The conjunction of two signals is not their sum. A new state arising from the simultaneous presence of A and B is, in general, not recoverable from A and B in isolation; it is a joint outcome with its own content. This rules out superposition as addition, ensemble outputs as averaged votes, and composition as linear mixing, replacing them with a model in which composition produces structure that was not previously anywhere in the system.

3. Formal Postconditions

The principles become operative only when expressible as predicates over system state. Let a Conjunction OS state be a triple:

```
sigma = (W, L, D)
W : multiset[Observation]      -- working registers
L : dict[Signature, Accumulator] -- latent register
D : list[Residue]              -- routed during this cycle
```

A cycle transforms `sigma_before` -> `sigma_after`. Each principle is a postcondition with a detection condition, a named violation, and a mandatory response.

P1. Non-Identity. For every pair (a,b) with `surface(a)=surface(b)`, structural hashes are equal. Violation: `surface-identical`, structurally distinct observations appear in W during COMPOSE. Response: raise `NonIdentityViolation` and abort before EMIT.

P2. Conservative Residue. `canonical_content(a)+canonical_content(b) = canonical_content(joint)+canonical_content(residue)`. Violation: residue is `None`, `untyped`, or silently compressed. Response: raise `UnroutedResidue`; the cycle does not reach SETTLE.

P3. Cyclic Closure. At SETTLE, W is empty and L contains only sub-threshold accumulators. Violation: W nonempty at SETTLE, or a ready accumulator undrained. Response: `phase="open"`; the next /cycle returns HTTP 423 until closure.

P4. Non-Additive Composition. `f(a,b).joint.structure` contains at least one component not pointwise derivable from a or b alone. Violation: a joint output componentwise reducible to its inputs. Response: a design-time invariant enforced by `merge_graphs`; the runtime surface is the joint marker edge.

The principles are no longer stances; they are predicates a reference implementation can check.

4. The Four-Phase Cycle

The postconditions are satisfied by a cyclic architecture with four phases: **SAMPLE**, **COMPOSE**, **EMIT**, **SETTLE**. One complete cycle is the unit of computation. Closure is enforced at the boundary: a cycle may not begin until the previous cycle has reported **SETTLE**.

SAMPLE. Probes are queried, outputs held in working registers without being combined.

COMPOSE. Working registers pass through the integrity filter. Surviving conjunctions form the joint outcome; conjunctions producing residue are routed to the latent register.

EMIT. The latent register is checked for a crystallization condition; ready accumulators discharge a self-contained artifact.

SETTLE. Working registers are zeroed, routed residue has reached its destination, any artifact is handed off, and the cycle reports phase="settled".

```
def cycle(probes, latent_register, prev_phase):
    if prev_phase != "settled": raise CycleOpen(prev_phase) # P3
    observations = [probe() for probe in probes] # SAMPLE
    joint, residue = integrity_filter(observations) # COMPOSE
    latent_register.route(residue) # EMIT (P2)
    emission = (latent_register.crystallize()
                if latent_register.ready_to_crystallize() else None)
    return {"joint":joint,"residue":residue,
            "emission":emission,"phase":"settled"} # SETTLE
```

The loop mutates no hidden in-process state and returns no gradient. Residue is handled exactly once. The only exits are a **SETTLE** return or a raised violation.

5. The Integrity Filter

The integrity filter enforces P1 and P4 at the **SAMPLE/COMPOSE** boundary. It is total over (Observation, Observation): it raises or returns, never None. The residue is typed; the joint carries a non-additive marker edge that prevents it from being pointwise reducible to its inputs.

```
def integrity_filter(a, b):
    if a.surface == b.surface and \
        canonical_hash(a.structure) != canonical_hash(b.structure):
        raise NonIdentityViolation(a, b) # P1
    joint = merge_graphs(a.structure, b.structure) # P4: + joint edge
    residue_payload = symmetric_difference(a.structure, b.structure)
    residue = Residue(canonical_hash(residue_payload),
                      residue_payload, classify_kind(residue_payload)) # P2
    return joint, residue
```

`canonical_hash` is SHA-256 of canonical CBOR (RFC 8949) — deterministic, order-independent, type-preserving. `merge_graphs` returns the overlay union plus one labelled ("joint", prov(s1), prov(s2)) edge. `symmetric_difference` returns components in exactly one input under canonical form. `classify_kind` assigns an equivalence class; it does not compress.

6. Residue Routing and the Latent Register

If Non-Identity forbids equating distinct states, how can the register accumulate anything? The objection conflates *identity* with *equivalence*. A structural signature commits to an equivalence relation at a chosen granularity: two residues sharing a signature share a *kind*, not an *identity*. The register accumulates across a kind; it never claims two distinct payloads are the same payload.

```

structural_signature(r) := SHA-256(canonical_cbor(r.payload))

class LatentRegister:
    def route(self, r):
        if r is None: raise UnroutedResidue          # P2
        acc = self.accumulators.setdefault(r.signature,
            Accumulator(r.signature, r.kind))
        acc.add(r)
    def exceeds_threshold(self): # adaptive: log-count, bounded entropy
        return math.log2(self.n + 1) >= K and self.entropy <= TAU

```

The adaptive threshold answers the explosion worry. A narrow stream of similar residues fills few signatures quickly; a broad stream fills many slowly. The emission rate is a function of the input's structural density, not a tuned hyperparameter. A register that never emits is reporting that its input has no stable kind. Accumulation is keyed on signature, not timestamp — the register learns patterns, it does not record events. Crystallization resets the accumulator that produced it; the register is non-monotonic.

7. Crystallization

An emitted artifact must satisfy the context-free emission invariant: for all distinct register contexts C_1, C_2 , $\text{decode}(\text{artifact}, C_1) = \text{decode}(\text{artifact}, C_2)$. The artifact carries its own signature, content, provenance tuple, and timestamp; an emission that depends on its producing register has not really left the system. An Artifact is functionally the same kind of object a probe produces, so artifacts can be fed to other instances across a network without a translation layer. Crystallization is the protocol unit of distributed deployments.

8. A Worked Cycle

Two probes — p_{lex} (lexical) and p_{sem} (semantic) — observe the same surface; thresholds $K = \log_2(4) = 2$, $\text{TAU} = 0.8$.

Cycles 1–2 (route). Two surface-equal observations agree on a shared key ("tokens") and diverge disjointly elsewhere (pos vs sem). This does *not* raise: the residue (lex/sem divergence) is typed, hashed to signature `8c4a...e2f1`, and routed. The accumulator reaches $n=2$; $\log_2(3) = 1.58 < K$, so no emission.

Cycle 3 (crystallize). A third matching pair brings $n=3$; $\log_2(4) = 2 \geq K$ and entropy $0.6 \leq \text{TAU}$. The accumulator emits a self-contained Artifact and resets — a discrete, non-monotonic event. The register at rest is where it began, plus one artifact.

Cycle 4 (violation). An injected pair has identical surface and shared key "tokens" but conflicting content on a shared slot ("pos": NOUN vs VERB). The filter raises `NonIdentityViolation`; the cycle aborts before `EMIT`; the latent register is untouched; the HTTP surface returns 409. The aborted cycle is terminal, not open, so the next request is accepted immediately.

A violation is the conflicting-shared-key case, not merely the case where structures differ.

9. Three Rails: Hanoi, Toffoli, and the Justification of Setup Cost

What makes the Tower of Hanoi solvable at all is the third peg. With two pegs the problem is impossible; with three it is *reversible from any state* — from any valid configuration the system reaches any other in at most $2^n - 1$ moves, the state graph is strongly connected, every move has an

inverse, and no state is a sink. The third peg is the ancilla — the rail that holds a displaced disk while another is moved. It is the reason the computation has a reverse at all.

The Toffoli gate has the same topology: two controls, one ancilla, three rails out. It is the minimum-arity gate that makes classical universal computation possible under the reversibility constraint (Toffoli, 1980; Fredkin and Toffoli, 1982). Two-input reversible gates cannot realize AND without losing information. Three is the first count at which a gate can be both universal and its own inverse — the minimum rail count at which a classical computation can be run backwards.

This is the structural reason the Toffoli gate appears in the commitment step. Density (the accumulator's readiness) and convergence (the cycle's routing completion) are the control rails; the potential artifact token is the ancilla. The gate flips the ancilla — committing the artifact — iff both controls are high. Because the operation is its own inverse, the ancilla history is in principle a replayable log of emissions: the system is reversible from any cycle boundary in the same sense a Hanoi configuration is.

Measured (v4). A port of the subtractive solver (the third peg is 3 — here — there, by exclusion) was run for $n = 1 \dots 12$. The decision cost — maximum subgoal-stack depth — equals n exactly ($= \log_2 N$) at every n , while path length is $2^n - 1$. Reversal was tested directly: reversing the move trace and flipping each move's direction recovers the original scramble for every n , with no replanning. The reversal is calculation-free *because the trace was kept*. The underlying mathematics is established — the Hanoi state graph is a Sierpiński gasket, and the exclusion rule with its reversibility is the classical iterative/Gray-code solution. The contribution is the reading of the third rail as the ancilla the commit step needs, not a new result about Hanoi.

The upfront cost is real: $2^n - 1$ moves; ancilla preparation per gate; per-cycle closure latency. Landauer's principle (1961) fixes the credit side: every irreversible bit erasure dissipates at least $k \cdot T \cdot \ln 2$ of heat, which a reversible computation does not pay — it pays a front-loaded gate-count overhead instead. Conjunction OS makes the same trade at the architecture level: in exchange for per-cycle cost it buys auditable emission history, crash-recoverable state, debuggable cycles with no silent loss, and structurally safe migration. An instance that runs one cycle has not justified its overhead; one that runs millions, with audit and migration and recovery, has.

10. System Surface and HTTP Semantics

The external surface mirrors the cycle: one request, one cycle, with the contract tightened for concurrency, duplication, and crash recovery.

```
POST /cycle Headers: Idempotency-Key: <uuid> (required)
- key matches prior request -> return stored response (no re-run)
- prev_phase != settled      -> 423 cycle_open
- NonIdentityViolation       -> 409, typed body
- otherwise                  -> 200, {joint, residue, emission, phase}
GET /state      -> phase, latent snapshot, ETag = sha256(canonical_state)
GET /artifacts -> self-contained emissions; idempotent; safe to retry
GET /stream    -> SSE; one event per phase transition; phase_sequence de-dup
```

The persistence layer is a single canonical document (JSON or CBOR) holding L. Every cycle is a read, an in-memory computation, an atomic write (rename-into-place). An append-only event ledger records phase transitions; on crash recovery, replay the ledger to the last SETTLE boundary, whose latent document is authoritative. The server process holds no hidden accumulation between requests.

11. Evaluation Metrics

Four metrics make the architecture falsifiable; each maps to a postcondition.

Residue Conservation Ratio (RCR) = $\text{bytes}(\text{canonical}(\text{routed_residue} \cup \text{joint_structure})) / \text{bytes}(\text{canonical}(\text{inputs}))$. Target 1.0; below 1.0 indicates silent loss (P2).

Non-Identity Violation Detection Rate (NIVDR) = $\text{detected} / \text{injected}$, against a stream with a known injection rate. Target 1.0; a falling value means the filter is mis-specified.

Cycle Closure Latency (CCL) = $t(\text{SETTLE}) - t(\text{SAMPLE})$, reported p50/p95/p99. The target is stability, not minimization; a rising p99 indicates accumulation without discharge.

Emission Fidelity (EF) = $\text{successful_decodes} / \text{total_emissions}$, feeding each artifact to a fresh independent instance. Target 1.0; below 1.0 violates the context-free emission invariant.

12. Verified Reference Implementations

Section 11 makes the architecture falsifiable in principle. This section reports the first executable checks of its gate-level primitives. Each was implemented from the architecture's own description and tested exhaustively (or, where the input space is large, by adversarial and randomized sampling). The aim is twofold: confirm the primitives do what the prose claims, and state precisely *where each coincides with an established construction* so the architecture's contribution is located correctly.

12.1 The helical Toffoli gate

The commitment primitive is realized as a 15-step helical decomposition, $H \rightarrow (\text{CNOT}, T, T^+) \times 13 \rightarrow H$, with phase, remainder, and shadow registers mapped onto the breath. Emulating the finite-state machine cycle-by-cycle and driving all eight (q0,q1,q2) inputs confirms: $q2_out = \text{NAND}(q0,q1)$ when the ancilla is 1 and $\text{AND}(q0,q1)$ when it is 0 (correct on all rows); the controls are preserved; and applying the gate twice returns the identity for every input — it is its own inverse. The all-equal inputs (000, 111) raise the $0 \neq 1$ violation flag, though the flag is observed, not enforced.

Established-computation note. This is the standard CCNOT / Toffoli gate and its textbook 15-gate Clifford+T decomposition. The verified contribution is not the gate but the consistent mapping of its discarded intermediate information onto typed remainder and shadow registers.

12.2 NAND ring adder (carry as routed remainder)

A single primitive — NAND — composed in a ring. Each breath computes a sum bit (a XOR b) and a remainder (the carry, a AND b); the remainder is *routed back* around the ring rather than discarded, and the cycle repeats until the remainder settles to zero (phase lock). This is the Conservative Residue and Cyclic Closure principles instantiated in arithmetic. Exhaustive testing confirms correctness:

Width	Correct (all pairs)	Avg breaths	$\log_2(\text{width})$	Worst
4	256 / 256	2.4	2.0	4
8	65,536 / 65,536	2.9	3.0	8
16	all	4.1	4.0	16
32	all (sampled)	5.2	5.0	32
64	all (sampled)	6.3	6.0	64

The average carry-resolution depth tracks $\log_2(\text{width})$; the worst case (an all-ones carry walk such as $(2^n-1)+1$) equals the width.

Established-computation note. This is a *variable-latency / carry-completion adder*. That average carry-chain length is $O(\log n)$ — and so a carry-completion design beats fixed $O(n)$ ripple latency on typical inputs — is classical, and is what motivates carry-lookahead and parallel-prefix adders. Within each breath all bit positions compute in parallel; the ring trades “sequential across n positions” for “parallel across positions, sequential across $\approx \log n$ breaths.” It does not beat ripple-carry in the worst case. The verified contribution is the derivation of this regime from the remainder-routing axiom, not a new bound.

Correction recorded. An earlier reference implementation of addition (PrimalMathEngine.add) computed the bitwise XOR of two NAND-transformed inputs and *discarded the carry* — returning the correct sum on only 2 of 256 input pairs, and being state-dependent rather than a pure function. The error sat exactly where the architecture's own axiom forbids it: the carry is the remainder, and the remainder was wasted. The ring adder above is the corrected version, in which the carry is routed until closure. The correction is reported here because surfacing a silent loss rather than absorbing it is the behavior the architecture prescribes.

12.3 Reversible adder (Toffoli + CNOT)

The three-rail reversibility argument of Section 9 is instantiated as a ripple-carry adder built only from Toffoli and CNOT gates. Tested exhaustively at widths 3–5: it computes $a + b$ (sum plus carry-out) correctly on every input; the carry ancilla returns to 0 for every input (phase lock); the input register is preserved; and running the gate sequence backward — each Toffoli/CNOT being its own inverse — recovers the exact initial state on every input. No bit is erased; the whole circuit is bijective, so its Landauer cost is zero. The price is gate count: a 5-bit add uses 10 Toffoli and 21 CNOT gates.

Established-computation note. This is the Cuccaro et al. (2004) ripple-carry adder, a near-minimal-ancilla reversible adder. “Reverse by running the gates backward, without recalculation” is the defining property of reversible logic (Bennett, 1973), not a new claim. The verified contribution is the identification of the adder's self-cleaning carry ancilla with the architecture's commit ancilla, and the explicit accounting of the gate-count overhead as the front-loaded cost Section 9 predicts.

12.4 The spine: the one genuinely open fork

The reversible adder exposes a decision the architecture cares about and that conventional reversible computing simply resolves one way. Cuccaro *uncomputes* its carry ancilla back to zero — clean, reusable, garbage-free. The architecture's latent register (the “spine”) proposes the opposite: *retain* the routed remainder as cross-epoch signal. The Hanoi result sharpens the stakes: there, reversal is free precisely *because the trace was kept*. Standard reversible computing always uncomputes; it never pauses at the choice. Whether retaining the spine buys something that uncomputing discards — auditable history, credit assignment, learning that survives migration — is the architecture's distinctive and still-unproven claim. It is the natural subject of the next round of measurement, and the metrics of Section 11 (especially EF and CCL) are the instruments for it.

13. Related Work and Limitations

Conjunction OS draws on several established lines of work without claiming strict inheritance. Reversible computing (Toffoli, 1980; Bennett, 1973) provides the gate-level discipline of Sections 9 and 12. The Landauer bound (1961) underwrites the Conservative Residue Principle. The Cuccaro et al. (2004) adder is the construction verified in Section 12.3, and carry-completion arithmetic — with its classical $O(\log n)$ average carry length — is the regime of Section 12.2. The Hanoi state graph's Sierpiński structure and the Gray-code reversibility of its solution are the established mathematics behind Section 9. Formally verified systems such as seL4 (Klein et al., 2009) set the standard for postcondition-level specification. Content-addressed programming (Unison) shows that structural hashes can serve as first-class identity without surface-level collapse. Flow-based programming (Morrison, 1994) anticipates routing data rather than mutating it in place. Conjunction OS is compatible with all of these and identical with none: its contribution is the constitutional framing — four refusals expressed as checkable postconditions — and the phasic cycle that enforces them, not novelty at the level of the gates or adders, which (as Section 12 states plainly) coincide with known constructions.

Limitations deserve explicit ownership. Phasic execution pays a per-cycle overhead that eager, streaming designs avoid; the architecture is not a drop-in replacement where raw throughput dominates. The verified primitives match — they do not beat — their established counterparts: the ring adder ties ripple-carry in the worst case, and the reversible adder pays a gate-count premium. The architecture's distinctive claim (Section 12.4) — that retaining the spine buys something uncomputing discards — is stated but not yet measured. Signature collision risk is bounded by the hash function; the state document should pin the hash name. Distributed crystallization across shards is an open question. A hardware instantiation — an FPGA or reversible-logic substrate that realizes the commit step as a literal Toffoli gate and measures thermodynamic overhead against the Landauer bound — remains future work, though the verified reference gate of Section 12.1 is a step toward it. Finally, the non-additive marker edge introduced by `merge_graphs` is a design convention, not a theorem.

14. Conclusion

Conjunction OS is an architecture organized around what it will not do. It will not collapse distinct states. It will not erase residue. It will not allow cycles to run open. It will not treat composition as addition. Each refusal removes a class of silent failure that is otherwise load-bearing in conventional systems.

The cost is that the system is cyclic rather than streaming, phasic rather than monotonic, and its outputs are discrete artifacts rather than continuous traces. The benefit is that every piece of information either finds a destination or surfaces as a violation — nothing falls quietly out of the record.

Version 4 adds the first executable evidence. The commitment gate, the carry-routing ring adder, the reversible adder, and the Hanoi reversal have been implemented and measured; they do what the prose claims. They also coincide, where claimed, with established computation — carry-completion arithmetic, the Cuccaro adder, the Sierpiński structure of Hanoi — and the paper now says so directly. What that leaves standing as genuinely the architecture's own is narrow and clear: the constitutional framing, the phasic cycle, and the unproven wager that retaining the spine buys something that uncomputing throws away. That wager is the proper subject of the next round of work — not what a newborn Conjunction OS instance is good for, but what the architecture becomes

after enough cycles have closed to make the reversibility, and the retained remainder, show what they are worth.